```
% ps_readinput_parameters
%
% --------------------------------------------------------------------
% File.............: ps_readinput_parameters.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% --------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%
% v1.7.2.8, Freek van Leijen
% - removed Natmo_iter, atmo_range, linelo, pixlo, filter_length
% - added breakpoint2, do_apriori_sidelobe_mask, do_aposteriori_sidelobe_mask
% - added gamma_threshold, psc_distribution, ts_atmo_filter,
% - added ts_atmo_filter_length, ts_noise_filter, ts_noise_filter_length
% v1.7.2.17, Freek van Leijen
% - added weighted_unwrap
%

param_file_content = textread(param_file,'%s');


% General parameters
% --------------------------------------------------------------------

max_mem_buffer = readinput('max_mem_buffer',param_file_content,[]);
% MB, maximal memory size of buffer

visible_plots = readinput('visible_plots',param_file_content,[]);
% make plots on screen? ('y' or 'n')

detail_plots = readinput('detail_plots',param_file_content,[]);
% make detail plots (e.g. time series)? ('y' or 'n')

processing_groups = readinput('processing_groups',param_file_content,[]);
% groups of functions to process, [] or vector of numbers
% 1 = read input
% 2 = ps selection
% 3 = detrend
% 4 = atmosphere
% 5 = densification
% 6 = deformation model
% 7 = output

run_mode = readinput('run_mode',param_file_content,[]);
% 'normal', 'debug' or 'validation'


% Project parameters
% --------------------------------------------------------------------
project_id = readinput('project_id',param_file_content,[]);

input_file = readinput('input_file',param_file_content,[]);

sensor = readinput('sensor',param_file_content,[]);
% 'ers' or 'asar'
```

```
orbit = readinput('orbit',param_file_content,[]);
% 'asc' or 'desc'

crop = readinput('crop',param_file_content,[]);
% [] or [l0,lN,p0,pN]

az_spacing = readinput('az_spacing',param_file_content,[]);
% [m], approximate azimuth spacing

r_spacing = readinput('r_spacing',param_file_content,[]);
% [m], approximate range spacing

slc_selection_input = readinput('slc_selection_input',param_file_content,[]);
% 'filename', [numbers] or []

ifg_selection_input = readinput('ifg_selection_input',param_file_content,[]);
% 'filename', [numbers] or []

ref_cn = readinput('ref_cn',param_file_content,[]);
% [azimuth_coordinate range_coordinate]
% or [] (random reference point)

Ncv = readinput('Ncv',param_file_content,[]);
% number of points for emperical calibration
% 0 = no emperical calibration

ps_method = readinput('ps_method',param_file_content,[]);
% 'perio' for ambiguity function/periodogram,
% 'ils' for integer least-squares or
% 'boot' for bootstrapping

psc_model = readinput('psc_model',param_file_content,[]);
% models to evaluate as alternative hypothesis

ps_model = readinput('ps_model',param_file_content,[]);
% models to evaluate as alternative hypothesis

final_model = readinput('final_model',param_file_content,[]);
% model (only one!) to use for unwrapped data
if final_model<2
  error('The final model should be larger than ''1''.');
end

breakpoint = readinput('breakpoint',param_file_content,[]);
% [] (empty),[14] (index in Btemp) or [11027] (orbitnumber),
% breakpoint in case of double or triple linear model

breakpoint2 = readinput('breakpoint2',param_file_content,[]);
% [] (empty),[14] (index in Btemp) or [11027] (orbitnumber),
% breakpoint in case of triple linear model, should be larger
% than first breakpoint

ens_coh_threshold = readinput('ens_coh_threshold',param_file_content,[]);
% ensemble coherence threshold

varfac_threshold = readinput('varfac_threshold',param_file_content,[]);
% variance factor threshold
% 3 is a reasonable number
% inf in case no selection on variance factor

detrend_method = readinput('detrend_method',param_file_content,[]);
% detrend based on psc, 'yes' or 'no'
```

```
output_format = readinput('output_format',param_file_content,[]);
% 1 = wgs84, radar original, radar gis coordinates
% 2 = wgs84, radar original coordinates
% 3 = wgs84, radar gis coordinates
% 4 = radar gis coordinates
% 5 = terrafirma S5

stc_min_max = readinput('stc_min_max',param_file_content,[]);
% minimum and maximum distance used for spatio-temporal consistency
% e.g. [50,250] [m]

do_apriori_sidelobe_mask =
readinput('do_apriori_sidelobe_mask',param_file_content,[]);
% 'yes' or 'no' or 'piers', amplitude based. For 'piers' additional
% parameters are needed, see Psc parameters

do_aposteriori_sidelobe_mask =
readinput('do_aposteriori_sidelobe_mask',param_file_content,[]);
% 'yes' or 'no', phase based



% Geocoding parameters
%---------------------------------------------------------------------
master_res = readinput('master_res',param_file_content,[]);
% 'master.res' file

ref_height = readinput('ref_height',param_file_content,[]);
% [m], height of reference point t.o.v. wgs84 ellipsoid,
% for Netherlands +/- 43 meters



% Psc parameters
% ---------------------------------------------------------------------

amplitude_calibration = readinput('amplitude_calibration',param_file_content,
[]);
% 'yes' or 'no'

psc_selection_method = readinput('psc_selection_method',param_file_content,[]);
% 'threshold' = above threshold, one per gridcell
% 'eachgrid' = one per gridcell
% 'coherence' = spatial coherence (based on method by A. Hooper)
% 'coherence_eachgrid' = spatial coherence first, followed by
% lowest amplitude dispersion in case spatial coherence not low enough

psc_selection_gridsize = readinput('psc_selection_gridsize',param_file_content,
[]);
% [m], grid size for psc selection

psc_threshold = readinput('psc_threshold',param_file_content,[]);
% threshold for amplitude dispersion

max_arc_length = readinput('max_arc_length',param_file_content,[]);
% [m]

network_method = readinput('network_method',param_file_content,[]);
% 'delaunay' = Delaunay,
% 'spider' = spider network (more redundant)

Ncon = readinput('Ncon',param_file_content,[]);
% minimum number of connections (arcs) per psc,
% (network_method 'spider' only)
```

```
Nparts = readinput('Nparts',param_file_content,[]);
% number of partitions of a full cycle to which the arcs are divided,
% should be one of the following nubmers: 2,4,6,8,10,12,16
% (network_method 'spider'  only)

Npsc_selections = readinput('Npsc_selections',param_file_content,[]);
% number of psc selections, normaly 1,
% but maybe more for validation

filename_water_mask = readinput('filename_water_mask',param_file_content,[]);
% 'filename' or []

gamma_threshold = readinput('gamma_threshold',param_file_content,[]);
% gamma (coherence) threshold for first order psc selection
% (A. Hooper method), e.g., 0.45

psc_distribution = readinput('psc_distribution',param_file_content,[]);
% psc distribution, 'uniform' (shifted grid) or 'nonuniform'
% (potentially close together)

weighted_unwrap = readinput('weighted_unwrap',param_file_content,[]);
% weighted unwrapping, 'yes' or 'no'


% for 'piers' apriori sidelobe detection
  livetime_threshold = readinput('livetime_threshold',param_file_content,[]);
  % livetime_threshold is percentage of slc's that has an amplitude peak

  peak_tolerance = readinput('peak_tolerance',param_file_content,[]);
  % include local near maxima


% Ps parameters
% ---------------------------------------------------------------------

psp_selection_method = readinput('psp_selection_method',param_file_content,[]);
% 'highamp' = high amplitude in minimal number of images
% 'ampdisp' = amplitude dispersion (higher threshold than for psc selection)

psp_threshold1 = readinput('psp_threshold1',param_file_content,[]);
% part of total slc's, recommendation = 0.65 or
% amplitude dispersion, recommendation = 0.4

psp_threshold2 = readinput('psp_threshold2',param_file_content,[]);
% [dB], for psp_selection_method = 'highamp', recommendation = -2 dB

ps_eval_method = readinput('ps_eval_method',param_file_content,[]);
% 'psp' = ps potential,
% 'whole' = whole image (overrules psp_selection_method)

Namp_disp_bins = readinput('Namp_disp_bins',param_file_content,[]);
% Number of amplitude dispersion bins used for final densification

Ndens_iterations = readinput('Ndens_iterations',param_file_content,[]);
% Number of densification iterations

densification_flag = readinput('densification_flag',param_file_content,[]);
% densification by region growing, 'yes' or 'no'

ps_area_of_interest = readinput('ps_area_of_interest',param_file_content,[]);
% 'filename', [az0,azN,r0,rN] or []

dens_method = readinput('dens_method',param_file_content,[]);
```

```
% 'orig' = original method
% 'adapt' = adaptive method with iterations based on amplitude
% prognosis, multiple ps_models should be specified to have an effect

dens_check = readinput('dens_check',param_file_content,[]);
% 'nocheck' = no check (only one arc estimated)
% '2of3' = 2 of 3 arcs correspond
% '3of3' = 3 of 3 arcs correspond (minimal number of false detections)
% 'mode_fix' = fix by mode of connecting arc estimates

Nest = readinput('Nest',param_file_content,[]);
% Number of connecting arc estimates during densifications in case
% of 'mode_fix' ('nocheck'-> default is 1, '2of3' or '3of3' ->
% default is 3)


% Stochastic model parameters
% ------------------------------------------------------------------------

std_param = readinput('std_param',param_file_content,[]);
% [topo master_atmo subpixel linear quadratic cubic periodic]
% [par1 par2 par3 ...], standard deviations of parameters,
% used to set the search space for unwrapping algorithms

defo_range = readinput('defo_range',param_file_content,[]);
% [m], expected range of spatial deformation correlation

weighting = readinput('weighting',param_file_content,[]);
% 'unw', 'weight' or 'vce'

ts_atmo_filter = readinput('ts_atmo_filter',param_file_content,[]);
% 'block' = filter atmo in ts with block filter
% 'triangle' = filter atmo in ts with triangle filter
% 'gaussian' = filter atmo in ts with gaussian filter
% 'prediction' = work in progress

ts_atmo_filter_length = readinput('ts_atmo_filter_length',param_file_content,
[]);
% the length of the atmo filter in years, should be larger than
% ts_noise_filter_length? 3x?

ts_noise_filter = readinput('ts_noise_filter',param_file_content,[]);
% 'block' = filter noise in ts with block filter
% 'triangle' = filter noise in ts with triangle filter
% 'gaussian' = filter noise in ts with gaussian filter
% 'prediction' = work in progress

ts_noise_filter_length = readinput('ts_noise_filter_length',param_file_content,
[]);
% the length of the noise filter in years



% Bowl parameters
%------------------------------------------------------------------------

defo_method = readinput('defo_method',param_file_content,[]);
% deformation model estimation method
% [] = no deformation model
% 0 = kriging
% 1 = bowl: r1, r, xc, yc, zc
% 2 = bowl: r1, r, zc
% 3 = bowl: r1, zc
```

```
xc0 = readinput('xc0',param_file_content,[]);
% initial range coordinate of the centre of the bowl

yc0 = readinput('yc0',param_file_content,[]);
% initial azimuth coordinate of the centre of the bowl

zc0 = readinput('zc0',param_file_content,[]);
% initial z coordinate of the centre of the bowl

r0 = readinput('r0',param_file_content,[]);
% initial radius of the bowl (size Nepoch)

r10 = readinput('r10',param_file_content,[]);
% initial max depth of the bowl (size Nepoch)

epoch = readinput('epoch',param_file_content,[]);
% r and r1 will be estimated for these epochs
```